

Math728J – Final Project

Let $\Omega = (0, 1)^2$. Consider numerical solution of the boundary value problem

$$\begin{cases} -u_{xx}(x, y) - u_{yy}(x, y) + b(x, y)u(x, y) = f(x, y), & (x, y) \in \Omega \\ u(x, y) = 0, & (x, y) \in \partial\Omega \end{cases} \quad (1)$$

where $b(x, y) = 1$. We let $u(x, y) = 10.0xy(1 - x)(1 - y)$ and then the right hand side f is determined correspondingly. We consider an uniform square grid with grid points (x_i, y_j) which are chosen to be equidistant:

$$x_i = ih, \quad y_j = jh, \quad i, j = 0, 1, \dots, N \quad (h = 1/N)$$

and the center difference scheme:

$$\frac{-U_{i-1,j} - U_{i,j-1} + 4U_{i,j} - U_{i+1,j} - U_{i,j+1}}{h^2} + b_{i,j}U_{i,j} = f_{i,j},$$

$$i, j = 1, 2, \dots, N - 1,$$

where $b_{i,j} = b(x_i, y_j)$ and $f_{i,j} = f(x_i, y_j)$, and

$$U_{i,j} \approx u(x_i, y_j), \quad i, j = 1, 2, \dots, N - 1$$

$$U_{i,0} = U_{i,N} = U_{0,i} = U_{N,i} = 0, \quad i, j = 0, 1, \dots, N.$$

Let $\mathbf{x}$ be the vector consisting of unknowns $U_{i,j}$ ($i, j = 1, 2, \dots, N - 1$) ordered row-wise (from left to right) and upward. Then, we get a linear system such as

$$\mathbf{Ax} = \mathbf{b}$$

where $\mathbf{A} \in \mathbb{R}^{(N-1)^2 \times (N-1)^2}$ is a sparse matrix and $\mathbf{b} \in \mathbb{R}^{(N-1)^2}$. Implement an MPI program to compute an approximate solution of (1) by a **parallel version** of Conjugate Gradient method, where the iteration is stopped when either more than $Kmax = 1000$ iterations are reached or the residual $\|\mathbf{r}_k\|_2 < \epsilon \|\mathbf{b}\|_2$ with $\epsilon = \frac{h^2}{100}$. Choose $\mathbf{x}_0 = 0$ as the initial value for the iterations.

Requirements:

- 2D block partitioning should be used for domain decomposition, and $\{U_{i,j}\}$, $\{b_{i,j}\}$, $\{f_{i,j}\}$ (also $\mathbf{A}$ if it is explicitly formed) should be distributed across all processes for optimal memory.
- Your code should be OK for general number of processes P and any valid process topology (for example: $P = 6 = 1 \times 6 = 2 \times 3 = 3 \times 2 = 6 \times 1$).

Results for hand-in:

- Describe your algorithm in details.
- Your code in paper and email.
- Measure and list the computation times for $P = 1, 2, 4, 6, 8, 9$ and corresponding process topologies in a table for each of $N = 100, 200, 400$.
- Find the optimal time for each P and plot the speedup curve based on the optimal times for each of $N = 100, 200, 400$.